

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

*I **P Madhavi/192372133** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Madhavi

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Student Registration System Using Java Servlet

Scenario

A college wants to develop an online student registration system. Students enter their details through an HTML form, and the submitted information is processed by a Java Servlet. The servlet should support both **GET** and **POST** methods.

1. Servlet-Based Application Architecture

A servlet-based web application follows a **client-server architecture**.

Architecture

Student / Browser

|
| HTTP Request
v

Web Server / Servlet Container

|
v

Java Servlet

|
| Processing
v

Business Logic / Database

|
| HTTP Response
v

Student / Browser

Main components

1. **Client/Browser**
 - Student enters registration details.
 - Sends an HTTP GET or POST request.
2. **Web Server**
 - Receives the request.
 - Forwards the request to the appropriate servlet.
3. **Servlet Container**
 - Manages servlet life cycle.

- Examples include Apache Tomcat.
- Creates servlet objects and invokes their methods.

4. **Servlet**

- Processes the student's request.
- Validates and processes form data.
- Generates an HTTP response.

5. **Database**

- Stores student registration information when required.

2. **HTML Registration Form**

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Student Registration</title>
```

```
</head>
```

```
<body>
```

```
<h2>Student Registration</h2>
```

```
<form action="StudentServlet" method="post">
```

Name:

```
<input type="text" name="name"><br><br>
```

Register Number:

```
<input type="text" name="regno"><br><br>
```

Email:

```
<input type="email" name="email"><br><br>
```

Department:

```
<input type="text" name="department"><br><br>
```

```
<input type="submit" value="Register">

</form>

</body>
</html>
```

3. Java Servlet Using GET and POST

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");

        out.println("<html><body>");
        out.println("<h2>GET Request</h2>");
        out.println("Name: " + name + "<br>");
```

```

        out.println("Register Number: " + regno);
        out.println("</body></html>");
    }

```

`@Override`

```

public void doPost(HttpServletRequest request,
                    HttpServletResponse response)
                    throws ServletException, IOException {

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    String name = request.getParameter("name");
    String regno = request.getParameter("regno");
    String email = request.getParameter("email");
    String department = request.getParameter("department");

    out.println("<html><body>");
    out.println("<h2>Student Registration Successful</h2>");
    out.println("Name: " + name + "<br>");
    out.println("Register Number: " + regno + "<br>");
    out.println("Email: " + email + "<br>");
    out.println("Department: " + department + "<br>");
    out.println("</body></html>");
}
}

```

Note: `jakarta.servlet.*` is used with newer Jakarta Servlet/Tomcat versions. Older Tomcat projects may use `javax.servlet.*`.

4. Servlet Life Cycle

The servlet life cycle consists mainly of:

1. init()

The init() method is called **once** when the servlet is created.

```
public void init() throws ServletException {  
    System.out.println("Servlet Initialized");  
}
```

It can be used for:

- Initializing resources
- Establishing configuration
- Loading required data

2. service()

The servlet container calls service() for every incoming request.

HTTP Request

```
|  
service()  
/  \  
GET  POST  
|    |  
doGet() doPost()
```

For HttpServlet, the service() method determines whether the request is GET, POST, etc., and invokes the appropriate method.

3. destroy()

Called once before the servlet is removed from service.

```
public void destroy() {  
    System.out.println("Servlet Destroyed");  
}
```

It can be used to:

- Close database connections
- Release resources
- Perform cleanup

Life Cycle Diagram

Servlet Loading

|

```

v
init()
|
v
service()
|
+-----> doGet()
|
+-----> doPost()
|
v
destroy()

```

5. GET vs POST

Feature	GET	POST
Data location	URL	Request body
Visibility	Data can appear in URL	Data not normally shown in URL
Suitable for	Searching/retrieving data	Form submission
Sensitive data	Not recommended	More appropriate
Data size	More limited	Can handle larger request bodies
Bookmarking	Possible	Generally not appropriate

Decision

POST is more appropriate for sensitive registration data because the submitted values are placed in the HTTP request body rather than directly in the URL.

However, **POST alone does not provide encryption**. Sensitive information should be transmitted over **HTTPS**, with proper validation and authentication.

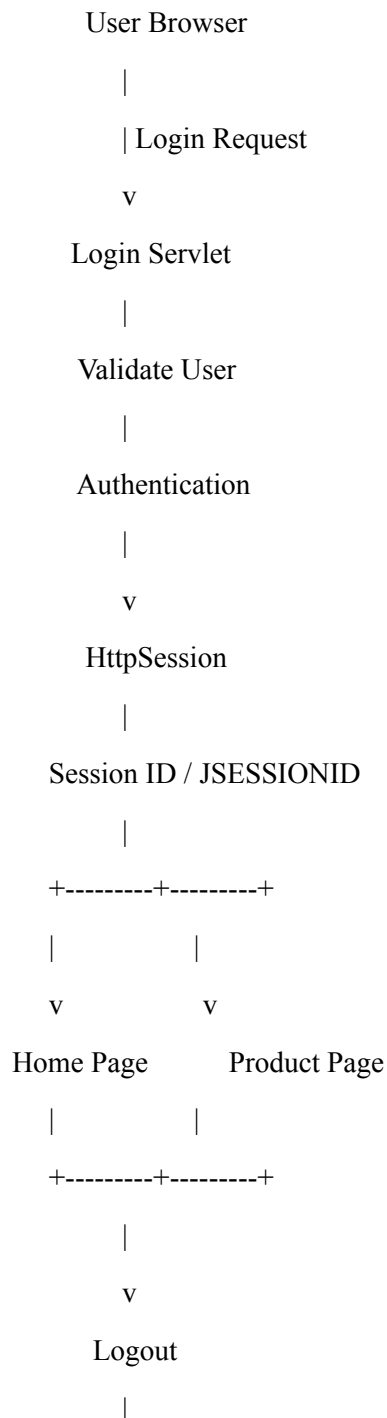
Question 2: Session Management Using HttpSession and Cookies

Scenario

An e-commerce website requires users to log in and remain authenticated while navigating between multiple pages.

The application can use **HttpSession** for server-side session management and **Cookies** for maintaining a session identifier.

1. Session Management Architecture



Session Invalidate

2. Login Servlet Using HttpSession

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");

        if ("admin".equals(username) &&
            "1234".equals(password)) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            response.sendRedirect("home");
        } else {
            response.getWriter().println("Invalid username or password");
        }
    }
}
```

```
}
```

3. Accessing Session on Another Page

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/HomeServlet")
public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        response.setContentType("text/html");

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            response.getWriter().println(
                "<h2>Welcome " + username + "</h2>"
            );

        } else {

            response.getWriter().println(
```

```

        "<h2>Please login first</h2>"
    );
}
}
}

```

4. Logout Servlet

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.getWriter().println(
            "<h2>You have been logged out.</h2>"
        );
    }
}

```

5. Cookies

A cookie is a small piece of information stored by the browser.

Example:

```
Cookie userCookie =
```

```
    new Cookie("username", "admin");
```

```
userCookie.setMaxAge(60 * 60);
```

```
response.addCookie(userCookie);
```

To read a cookie:

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {
```

```
    for (Cookie cookie : cookies) {
```

```
        if ("username".equals(cookie.getName())) {
```

```
            String username = cookie.getValue();
```

```
            response.getWriter().println(
```

```
                "Welcome " + username
```

```
            );
```

```
        }
```

```
    }
```

```
}
```

6. How Session Tracking Works

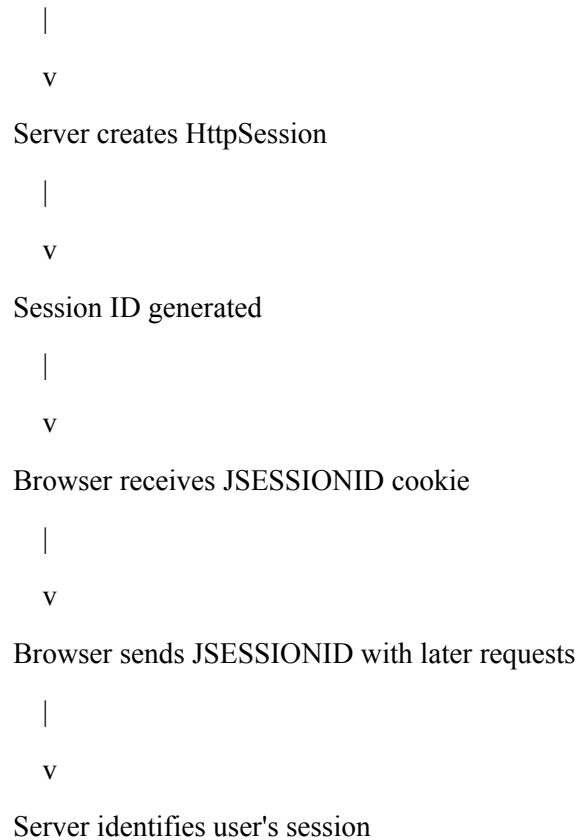
When a user logs in:

User Login

|

v

Server validates credentials



The actual session data is maintained on the **server**, while the browser normally stores the session identifier.

7. Session vs Cookies

Feature	HttpSession	Cookies
Storage	Mainly server-side	Client/browser
Data sent	Session ID	Cookie values
Security	Better for server-side state	Depends on cookie contents/settings
Storage capacity	Larger server-side storage	Limited
Lifetime	Configurable	Can be session or persistent
Common use	Login/session state	Preferences/session identifiers

Decision

For an authenticated website, **HttpSession is preferable for maintaining login state** because sensitive session information can remain on the server.

Cookies can still be used to carry the **session identifier**, but cookies should be configured securely, for example with HttpOnly, Secure, and an appropriate SameSite policy.

Question 3: JDBC-Based E-Commerce Product System

Scenario

An e-commerce platform needs to store and retrieve product details using a Java web application. The application uses **JDBC (Java Database Connectivity)** to communicate with the database.

1. JDBC Architecture

Java Web Application

|

v

JDBC API

|

v

JDBC Driver Manager

|

v

JDBC Driver

|

v

Database

|

v

SQL Queries / Results

Main components

1. Java Application

- Sends requests to the database.

2. JDBC API

- Provides classes and interfaces for database communication.

3. DriverManager

- Manages database drivers.

4. **JDBC Driver**

- Provides communication between Java and the specific database.

5. **Database**

- Stores product information.

2. Database Table

Example MySQL table:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE products (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE,  
    quantity INT  
);
```

3. **JDBC Connection and Insert Product**

```
import java.sql.*;
```

```
public class ProductDAO {
```

```
    private static final String URL =  
        "jdbc:mysql://localhost:3306/ecommerce";
```

```
    private static final String USER = "root";  
    private static final String PASSWORD = "root";
```

```
    public void insertProduct(String name,  
        double price,
```



```

        int quantity) {

String sql =
    "INSERT INTO products(name, price, quantity) VALUES (?, ?, ?)";

try (Connection con =
    DriverManager.getConnection(URL, USER, PASSWORD);

    PreparedStatement ps =
        con.prepareStatement(sql)) {

    ps.setString(1, name);
    ps.setDouble(2, price);
    ps.setInt(3, quantity);

    int rows = ps.executeUpdate();

    System.out.println(
        rows + " product inserted successfully."
    );

} catch (SQLException e) {

    System.out.println(
        "Database error: " + e.getMessage()
    );
}
}
}

```

4. Retrieve Product Details

```
import java.sql.*;
```

```
public class ProductRetrieve {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/ecommerce";

        String user = "root";
        String password = "root";

        String sql =
            "SELECT id, name, price, quantity FROM products";

        try (Connection con =
            DriverManager.getConnection(url, user, password);

            PreparedStatement ps =
                con.prepareStatement(sql);

            ResultSet rs =
                ps.executeQuery()) {

            while (rs.next()) {

                int id = rs.getInt("id");
                String name = rs.getString("name");
                double price = rs.getDouble("price");
                int quantity = rs.getInt("quantity");

                System.out.println(
                    id + " | " + name + " | " +
```

```

        price + " | " + quantity
    );
}

} catch (SQLException e) {

    System.out.println(
        "Database error: " + e.getMessage()
    );
}
}
}

```

5. Steps Involved in JDBC Connectivity

Step 1: Import JDBC packages

```
import java.sql.*;
```

Step 2: Establish database connection

```

Connection con =
    DriverManager.getConnection(url, user, password);

```

Step 3: Create SQL statement

For parameterized queries, use:

```

PreparedStatement ps =
    con.prepareStatement(sql);

```

Step 4: Set parameters

```

ps.setString(1, name);
ps.setDouble(2, price);
ps.setInt(3, quantity);

```

Step 5: Execute SQL query

For INSERT, UPDATE, DELETE:

```
ps.executeUpdate();
```

For SELECT:

```
ResultSet rs = ps.executeQuery();
```

Step 6: Process results

```
while (rs.next()) {  
    System.out.println(rs.getString("name"));  
}
```

Step 7: Close resources

Using **try-with-resources** automatically closes the connection, statement, and result set.

6. Exception Handling

Database operations can produce exceptions such as:

- Invalid database URL
- Incorrect username/password
- Database unavailable
- SQL syntax error
- Constraint violation
- Connection failure

Therefore, JDBC operations should be handled using:

```
try {  
    // JDBC operations  
}  
catch (SQLException e) {  
    System.out.println("Database error: "  
        + e.getMessage());  
}
```

Using PreparedStatement is also preferred over constructing SQL strings by concatenation because it helps prevent **SQL injection**.

Overall Decision Making

Scenario	Recommended Technology	Reason
Student form submission	POST Servlet	Suitable for form data and sensitive fields
Data retrieval	GET Servlet	Appropriate for retrieving non-sensitive information

User authentication	HttpSession	Maintains server-side login state
Session identification	Secure Cookie/JSESSIONID	Identifies the user's session
Product storage	JDBC	Provides Java-database connectivity
SQL execution	PreparedStatement	Safer parameterized SQL
Database errors	SQLException handling	Prevents application failure